

Introduction to AlgebraicABMs.jl

Building agent-based models with algebraic rewriting

Simon Frost

2026-03-30

1 Overview

AlgebraicABMs.jl is a Julia framework for building rigorous agent-based models (ABMs) using the mathematics of *algebraic rewriting*. Rather than writing imperative update rules, you declaratively specify:

1. **Schemas:** The types of agents and their relationships (as C-sets)
2. **Rules:** How the state transforms (as double-pushout rewriting rules)
3. **Timers:** When events happen (as hazard rate functions)

The framework then handles event scheduling, pattern matching, and state updates automatically using the theory of *competing clocks* on *incremental homomorphism sets*.

This vignette introduces the core concepts through a classic **SIR (Susceptible–Infected–Recovered)** epidemiological model.

2 Setup

```
using AlgebraicABMs
using Catlab, AlgebraicRewriting
using Distributions: Exponential
using Random
Random.seed!(42)

TaskLocalRNG()
```

3 Step 1: Define the Schema

A **schema** describes the types of entities and their relationships. For an SIR model on a contact graph, we need:

- **Vertices (V):** individuals in the population
- **Edges (E):** contacts between individuals
- **Infection status:** tracked via subsets of vertices

We use the built-in **Graph** schema from Catlab, which has objects **V** and **E** with homs **src**, **tgt** : **E** → **V**.

In an SIR model, the “S”, “I”, and “R” compartments are represented structurally: the *state* is an annotated graph where specific vertices are susceptible, infected, or recovered.

```
# We work with the standard Graph schema from Catlab
# V = vertices (individuals), E = edges (contacts)
# Compartments are tracked by the presence/absence of individuals
# in different parts of the ASet
```

```
# Let's understand the Graph schema
println("Graph schema: vertices (V) and edges (E) with src, tgt homs")
println("  Objects: V, E")
println("  Homs: src : E → V, tgt : E → V")

Graph schema: vertices (V) and edges (E) with src, tgt homs
  Objects: V, E
  Homs: src : E → V, tgt : E → V
```

4 Step 2: Define the Initial State

The initial state is an **ACSet** (attributed C-set) — a database-like structure conforming to the schema. We create a complete contact graph with some individuals initially infected.

```
# Create a small population: 10 individuals, fully connected
n_pop = 10
n_infected = 2

# Build the initial graph: complete graph on n_pop vertices
init = @acset Graph begin
  V = n_pop
  E = n_pop * (n_pop - 1)
  src = vcat([[i for _ in 1:(n_pop-1)] for i in 1:n_pop]...)
  tgt = vcat([[j for j in 1:n_pop if j != i] for i in 1:n_pop]...)
end

println("Population: $(nparts(init, :V)) individuals")
println("Contacts: $(nparts(init, :E)) directed edges")

Population: 10 individuals
Contacts: 90 directed edges
```

5 Step 3: Define Rewrite Rules

Each event in the model is a **rewrite rule** — a span $L \leftarrow I \rightarrow R$ specifying:

- **L** (left): the pattern to match in the current state
- **I** (interface): what is preserved
- **R** (right): the replacement pattern

5.1 Infection Rule

An infection event requires a contact edge between two individuals. One individual is removed (becomes infected — in our simple model, infection “consumes” a susceptible vertex and its edges are redistributed).

For simplicity, let’s use the minimal approach: infection *removes an edge* (breaking a contact), and recovery *removes a vertex* (individual leaves the population).

```
# Simple model: each edge represents a potential transmission
# Infection removes an edge (contact used up)
L_inf = @acset Graph begin V=2; E=1; src=[1]; tgt=[2] end # edge pattern
I_inf = Graph(2) # keep both vertices
R_inf = Graph(2) # no edge in result

infection = ABMRule(:Infection,
```

```

    Rule(homomorphism(I_inf, L_inf; initial=(V=[1,2],)),
        homomorphism(I_inf, R_inf; initial=(V=[1,2],))),
    ContinuousHazard(0.3))

println("Infection rule: removes an edge at rate 0.3")
Infection rule: removes an edge at rate 0.3

```

5.2 Recovery Rule

Recovery removes a self-loop (if we track infection status via self-loops) or, more simply, removes a vertex from the graph.

```

# Recovery: create a self-loop (marks vertex as recovered)
L_rec = Graph(1) # single vertex
I_rec = Graph(0) # nothing preserved
R_rec = Graph(0) # vertex removed

recovery = ABMRule(:Recovery,
    Rule(homomorphism(I_rec, L_rec),
        id(I_rec)),
    ContinuousHazard(Exponential(5.0))) # mean recovery time = 5

println("Recovery rule: removes a vertex with mean time 5.0")
Recovery rule: removes a vertex with mean time 5.0

```

6 Step 4: Build and Run the ABM

Combine rules into an ABM and run the simulation.

```

abm = ABM([infection, recovery])

println("ABM with $(length(abm)) rules:")
for (i, rule) in enumerate(abm.rules)
    println(" Rule $i: $(rule.name)")
end

ABM with 2 rules:
  Rule 1: Infection
  Rule 2: Recovery

# Run the simulation
result = run!(abm, deepcopy(init); maxtime=20.0)

println("\nSimulation complete:")
println(" Events fired: $(length(result))")
println(" Time span: 0.0 to $(round(result.events[end][1], digits=3))")

[ Info: Stochastic scheduling algorithm ran out of events

Simulation complete:
  Events fired: 100
  Time span: 0.0 to 9.892

```

7 Step 5: Analyze the Trajectory

The `Traj` object records every event that occurred.

```
# Event timeline
println("Event log (first 10 events):")
for (i, (t, rule_id, rule_name, _)) in enumerate(result.events)
    i > 10 && break
    println("  t=$(round(t, digits=3)): $(rule_name)")
end

Event log (first 10 events):
  t=0.001: Infection
  t=0.005: Infection
  t=0.005: Infection
  t=0.013: Infection
  t=0.017: Infection
  t=0.03: Infection
  t=0.041: Infection
  t=0.044: Infection
  t=0.048: Infection
  t=0.055: Infection

# Track population size over time using the trajectory history
if !isempty(result.hist)
    println("\nState sizes over time:")
    println("  Initial vertices: $(nparts(result.init, :V))")
    for (i, span) in enumerate(result.hist)
        final_state = codom(right(span))
        t = result.events[i][1]
        nv = nparts(final_state, :V)
        ne = nparts(final_state, :E)
        i <= 15 && println("  After event $i (t=$(round(t,digits=2))): V=$nv, E=$ne")
    end
end

end
```

```
State sizes over time:
  Initial vertices: 10
  After event 1 (t=0.0): V=10, E=89
  After event 2 (t=0.0): V=10, E=88
  After event 3 (t=0.01): V=10, E=87
  After event 4 (t=0.01): V=10, E=86
  After event 5 (t=0.02): V=10, E=85
  After event 6 (t=0.03): V=10, E=84
  After event 7 (t=0.04): V=10, E=83
  After event 8 (t=0.04): V=10, E=82
  After event 9 (t=0.05): V=10, E=81
  After event 10 (t=0.05): V=10, E=80
  After event 11 (t=0.06): V=10, E=79
  After event 12 (t=0.06): V=10, E=78
  After event 13 (t=0.07): V=10, E=77
  After event 14 (t=0.07): V=10, E=76
  After event 15 (t=0.08): V=10, E=75
```

8 Step 6: Using Observables

Observables track pattern match counts over time without storing full state snapshots.

```
# Count vertices and edges over time
obs_v = Observable(:Vertices, Graph(1)) # count vertices
obs_e = Observable(:Edges, @acset Graph begin V=2; E=1; src=[1]; tgt=[2] end) # count edges

result2 = run!(abm, deepcopy(init);
    maxtime=20.0,
    observables=[obs_v, obs_e],
    save_every=1.0)

println("Observable snapshots recorded: $(length(result2.snapshots))")
if !isempty(result2.snapshots)
    println("\nPopulation trajectory (sampled every 1.0 time units):")
    for (t, counts) in result2.snapshots
        println("  t=$(round(t, digits=1)): Vertices=$(counts[:Vertices]), Edge-patterns=$(counts[:Edges])")
    end
end

[ Info: Stochastic scheduling algorithm ran out of events
Observable snapshots recorded: 2

Population trajectory (sampled every 1.0 time units):
  t=0.0: Vertices=10, Edge-patterns=90
  t=1.0: Vertices=10, Edge-patterns=1
```

9 Step 7: Vertex Creation Rules

A key feature of algebraic ABMs is **vertex creation** — rules that add new agents to the system. Let's add a birth rule.

```
# Birth: create a new vertex (connected to nothing initially)
birth = ABMRule(:Birth,
    Rule(id(Graph()), # L = I =
        create(ob{terminal(Graph)})), # R = single vertex
    ContinuousHazard(0.5))

abm_birth = ABM([infection, recovery, birth])
result3 = run!(abm_birth, deepcopy(init); maxtime=15.0)

println("With births: $(length(result3)) events over $(round(result3.events[end][1], digits=2)) time units")

# Count events by type
event_counts = Dict{String, Int}()
for (_, _, name, _) in result3.events
    event_counts[name] = get(event_counts, name, 0) + 1
end
println("Event counts: ", event_counts)

With births: 178 events over 15.19 time units
Event counts: Dict("Birth" => 31, "Infection" => 120, "Recovery" => 27)
```

10 Summary

This vignette demonstrated the core AlgebraicABMs workflow:

Concept	AlgebraicABMs	Description
Schema	@present, Graph	Types of agents and relationships
State	@acset	Current population as a database
Rule	ABMRule(Rule(...), timer)	How the state changes
Timer	ContinuousHazard, DiscreteHazard	When events happen
ABM	ABM([rules...])	Collection of competing rules
Simulation	run!(abm, init)	Stochastic simulation via competing clocks
Trajectory	Traj	Full event history
Observables	Observable	Efficient state tracking

In subsequent vignettes, we explore more sophisticated timer types, spatial structure, interventions, hybrid ODE models, and calibration.